



GitHub Actions & CI/CD

Concept • Workflow YAML • Secrets • Caching • Act

By: Pranpaveen Laykaviriyakul

GitHub Actions คืออะไร?

Platform สำหรับทำ CI/CD และ automation ที่รวมอยู่ใน GitHub ตั้งแต่แรก
เขียน workflow เป็น YAML ไฟล์ เก็บไว้ใน repository เดียวกันกับโค้ด

A blue square icon with the word "FREE" in white capital letters.

Free Tier

Public repos ฟรีไม่จำกัด
Private repos 2,000 นาที/
เดือน



Marketplace

Actions สำเร็จรูปกว่า
20,000+
github.com/marketplace



Tight Integration

ทำงานกับ Issues, PR,
Releases
โดยตรงโดยไม่ต้องตั้งค่า
webhook



Matrix Build

ทดสอบบน OS/Python
version
หลายชุดพร้อมกัน

Components ของ GitHub Actions

Event / Trigger

สิ่งที่ทำให้ workflow รัน เช่น push, pull_request, schedule, workflow_dispatch

Workflow

ไฟล์ YAML ใน .github/workflows/ — กำหนด pipeline ทั้งหมด 1 repo มีได้หลาย workflow

Job

หน่วยงานหลักใน workflow — แต่ละ job รันบน runner แยกกัน สามารถ parallel หรือ sequential ได้

Step

คำสั่งที่รันใน job — อาจเป็น shell command (run:) หรือ action สำเร็จรูป (uses:)

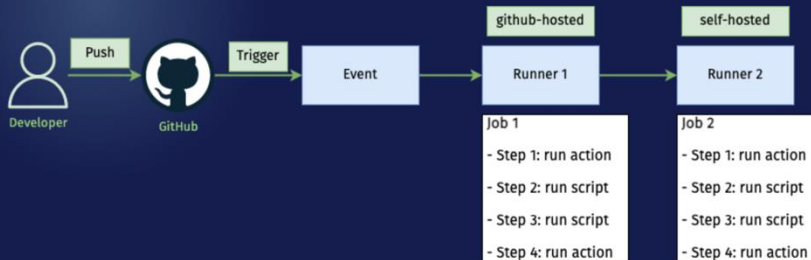
Action

โค้ดสำเร็จรูปที่ package มาแล้ว เช่น actions/checkout, actions/setup-python

SECTION

01

How GitHub Actions Work?



Workflow YAML

โครงสร้างและ syntax ของ workflow file

โครงสร้าง Workflow YAML

 [.github/workflows/01-echo.yaml](#) (จาก github.com/layel2/basic-github-action)

```
name: GitHub Actions Demo           # ชื่อที่แสดงใน Actions tab
run-name: ${github.actor} is testing out GitHub Actions 🚀
on: [push, workflow_dispatch]       # trigger ทุก push และ manual run

jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest           # รันบน Ubuntu runner ของ GitHub
    steps:
      - run: echo "🎉 Triggered by ${github.event_name} event."
      - run: echo "🖥️ Running on ${runner.os} server."
      - run: echo "🔗 Branch: ${github.ref}"

      - name: Check out repository code
        uses: actions/checkout@v4    # clone repo ลงบน runner

      - name: List files in the repository
        run: |
          ls ${github.workspace}     # | = multiline command

      - run: echo "🍏 Job status: ${job.status}."
```

Triggers — on: keyword

ใน repo basic-github-action มีตัวอย่าง trigger 3 แบบ:

📁 จาก repo จริง

```
# 01-echo.yaml — push ทุก branch + manual
on: [push, workflow_dispatch]

# 02-echo-2.yaml — push ทุก branch
on: [push]

# 03-build-test.yaml — push main only
on:
  push:
    branches:
      - main
  workflow_dispatch:
```

📖 Triggers อื่นๆ ที่ใช้อยู่

```
on:
  pull_request:           # เมื่อเปิด PR
    branches: [ main ]

  schedule:               # Cron Job
    - cron: '0 8 * * 1-5'

  release:
    types: [published]    # เมื่อ publish release
```

💡 workflow_dispatch — สำคัญมากสำหรับ manual trigger ผ่าน Actions UI หรือ API
เหมาะสำหรับ deploy ที่ต้องการ approve ก่อนรัน

Jobs, Steps & Runners

Jobs รันแบบ Parallel โดย default — ถ้าต้องการ sequential ต้องใช้ needs:

Jobs:

```
test:                                # Job 1 – รัน pytest
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: docker build -t testapi .
    - run: docker run testapi pytest

build:                                # Job 2 – รอ test ผ่านก่อน
  needs: test                          # ← sequential dependency
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: docker build -t testapi .

deploy:
  needs: [test, build]                 # รอทั้ง 2 job ก่อน
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main' # รันเฉพาะ main
  steps:
    - run: echo "Deploying..."
```

SECTION

02

Secrets & Env Vars

จัดการ credentials อย่างปลอดภัยใน workflow

Secrets — เก็บ credential อย่างปลอดภัย

Secret คือค่าที่ sensitive เช่น API key, password, token
GitHub เข้ารหัสก่อน store และ mask ใน log อัตโนมัติ

วิธีเพิ่ม Secret

Repo → Settings → Secrets and variables → Actions → New repository secret

ชื่อ Secret ควรเป็น UPPER_SNAKE_CASE

ใน basic-github-action ใช้: DOCKER_USERNAME, DOCKER_PASSWORD

วิธีใช้ใน workflow

```
- name: Login to Docker Hub
  uses: docker/login-action@v1
  with:
    username: ${ secrets.DOCKER_USERNAME }
    password: ${ secrets.DOCKER_PASSWORD }
```

⚠ สิ่งที่ต้องระวัง

```
# - Secret จะเป็น *** ใน log เสมอ
# - Fork PR จาก outsider ไม่ได้รับ secrets
# - ไม่ควร echo secret ออกมา
# - ใช้ Environment secrets สำหรับ deploy
#   แยกตาม staging / production
```

```
# Tip: ใช้ GITHUB_TOKEN สำหรับ GitHub API
# มีให้อัตโนมัติ ไม่ต้องสร้างเอง
- run: gh release create v1.0.0
  env:
```

Environment Variables ใน Workflow

กำหนด env vars ได้ 3 ระดับ: workflow, job, step — ระดับที่แคบกว่า override ระดับที่กว้างกว่า

```
env:                                # ระดับ Workflow (ทุก job เห็น)
  PYTHON_ENV: production
  APP_VERSION: ${github.sha}       # ใช้ context variables ได้

jobs:
  build-and-test:
    env:                            # ระดับ Job
      IMAGE_TAG: dev-${github.run_number}
    steps:
      - name: Build
        env:                        # ระดับ Step
          BUILD_TARGET: testapi
        run: docker build -t $BUILD_TARGET .

      - name: Test
        run: |
          echo "Testing version $APP_VERSION"
          docker run testapi pytest -v
        env:
          API_KEY: ${secrets.API_KEY} # ผสม secret + env
```

Context ที่ใช้บ่อย: `${github.actor}` `${github.ref}` `${github.sha}` `${runner.os}`

SECTION

03

Caching & Artifacts

เร่ง build ด้วย cache และส่งต่อ build output

Caching — เร่ง Workflow ด้วย Cache

โดยไม่มี cache → pip install ใช้เวลา 1-2 นาทีทุกครั้ง

ด้วย cache → ครั้งแรก 1-2 นาที ครั้งต่อไป < 10 วินาที ✓

```
- name: Cache pip packages
  uses: actions/cache@v4
  with:
    path: ~/.cache/pip          # folder ที่ cache
    key: ${ runner.os }-pip-${ hashFiles('**/requirements.txt') }
    restore-keys: |
      ${ runner.os }-pip-      # fallback key

- name: Install dependencies
  run: pip install -r requirements.txt

# — หรือใช้ shortcut ของ setup-python —————
- uses: actions/setup-python@v5
  with:
    python-version: '3.11'
    cache: 'pip'                # ← cache pip อัตโนมัติ!
```

Cache Key: ถ้า key ตรง → restore จาก cache, ถ้าไม่ตรง → รัน step แล้ว save cache ใหม่

Cache Limit: สูงสุด 10 GB ต่อ repo, cache ที่ไม่ได้ใช้ 7 วัน ถูกลบอัตโนมัติ

Artifacts — ส่งต่อ Build Output

Artifact คือไฟล์ที่ผลิตจาก workflow เช่น build output, test reports, binary files สามารถ download จาก GitHub UI หรือใช้ใน job อื่นใน workflow เดียวกัน

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build & Test
        run: |
          docker build -t testapi .
          docker run testapi pytest --cov=. --cov-report=xml
          docker cp $(docker ps -lq):/app/coverage.xml .

      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage.xml
          retention-days: 7

  report:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Download coverage report
        uses: actions/download-artifact@v4
        with:
          name: coverage-report
      - run: cat coverage.xml
```

SECTION

04

CI/CD Example

ตัวอย่าง pipeline สำหรับ Python (FastAPI) + Docker

CI Pipeline — Build & Test Docker

 `.github/workflows/03-build-test.yaml` (จาก repo จริง)

```
name: Build and Test
on:
  push:
    branches: [ main ]
  workflow_dispatch:

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Build Docker image
        run: docker build -t testapi .

      - name: Run tests
        run: docker run testapi pytest
```

① Checkout

clone repo ลงบน
GitHub runner

② Build Image

build Dockerfile
ได้ image testapi

③ Run pytest

test ใน container
environment จริง

workflow_dispatch

รัน manual ได้
ผ่าน Actions UI

CD Pipeline — Build, Test & Push to Docker Hub

 `.github/workflows/04-build-test-push.yaml` (จาก repo จริง)

```
name: Build and Test
on:
  push:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Build Docker image
        run: docker build -t testapi .

      - name: Run tests
        run: docker run testapi pytest

      - name: Login to Docker Hub          # ใช้ Secrets ที่ตั้งไว้ใน repo
        uses: docker/login-action@v1
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }

      - name: Tag Docker image
        env:
          IMAGE_TAG: dev-${ github.run_number } # เช่น dev-42
        run: docker tag testapi ${ secrets.DOCKER_USERNAME }/testapi:$IMAGE_TAG

      - name: Push Docker image to Docker Hub
        env:
          IMAGE_TAG: dev-${ github.run_number }
        run: docker push ${ secrets.DOCKER_USERNAME }/testapi:$IMAGE_TAG
```

① Build & Test

ทดสอบก่อน push เสมอ

② Secrets

DOCKER_USERNAME
DOCKER_PASSWORD
→ ตั้งใน repo Settings

③ Auto Tag

dev-{run_number}
เช่น dev-1, dev-2, dev-3

④ Push to Hub

\$username/testapi:dev-42

SECTION

05

Act — Local Runner

ทดสอบ workflow บนเครื่องก่อน push

Act คืออะไร?

Act เป็น open-source tool ที่รัน GitHub Actions บนเครื่องตัวเอง
ช่วยประหยัดเวลาจากการ push ทุกครั้งเพื่อทดสอบ workflow



เร็วกว่า

ทดสอบทันทีบน local ไม่
ต้องรอ GitHub runner



ประหยัด

ไม่ใช้ minutes quota ของ
GitHub



ปลอดภัย

ทดสอบ workflow ที่มี
secret ได้โดยใช้ .env file



Debug ง่าย

ดู output real-time, แก้แล้ว
ทดสอบซ้ำได้เลย

GitHub: <https://github.com/nektos/act> | Install: <https://nektosact.com/installation/>

ติดตั้งและตั้งค่า Act

ติดตั้ง

```
# macOS
brew install act

# Windows (winget)
winget install nektos.act

# หรือ download binary จาก GitHub Releases
```

ตั้งค่า

```
# สร้าง .actrc ใน home directory
echo '-P ubuntu-latest=cattthehacker/ubuntu:act-latest' >> ~/.actrc

# Secrets สำหรับ repo basic-github-action
# สร้างไฟล์ .secrets ใน root ของโปรเจกต์
DOCKER_USERNAME=your-dockerhub-username
DOCKER_PASSWORD=your-dockerhub-password
# เพิ่ม .secrets ใน .gitignore!
```

ใช้งาน Act กับ basic-github-action

ลองใช้ Act กับ workflows ใน repo layer2/basic-github-action:

```
# ดู workflows ทั้งหมดใน repo
act --list
# Workflow 1: GitHub Actions Demo          (01-echo.yaml)
# Workflow 2: GitHub-Actions-Demo2         (02-echo-2.yaml)
# Workflow 3: Build and Test                (03-build-test.yaml)
# Workflow 4: Build and Test                (04-build-test-push.yaml)

# รัน 01-echo.yaml (push event)
act push

# รัน 03-build-test.yaml (build + test Docker)
act push --secret-file .secrets

# รัน 04-build-test-push.yaml (full CD pipeline)
act push --secret-file .secrets # ต้องมี DOCKER_USERNAME + DOCKER_PASSWORD

# รัน workflow_dispatch (manual trigger)
act workflow_dispatch

# ดู dry run ก่อนรันจริง
act -n

# verbose - ดู output แบบละเอียด
```

Workshop & Q&A

Clone: <https://github.com/layel2/basic-github-action>

ลองทำ: แก้ workflow → act → push → ดู Actions tab → สร้าง Release

References

<https://docs.github.com/en/actions>

<https://github.com/marketplace?type=actions>

<https://nektosact.com/usage/index.html>

<https://github.com/nektos/act>

<https://github.com/layel2/basic-github-action>